

QUANT FINANCE TERMINAL ·  
v2.046

SYS://CURRICULUM/D033.MD · PHASE 2 ●  
LIVE

LECTURE · 量化金融 365

# DAY 033 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

## NumPy 进阶:矩阵运算

// Linear Algebra

KEY CONCEPTS · 三大要点

- ▶ @ 运算符
- ▶ linalg
- ▶ 特征分解

01 · WHY THIS LESSON · 这节课为什么重要

## 本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

今天讲 NumPy 进阶里最重要的1 块 — 矩阵运算。上一节我们学了 NumPy 的基础,把数据想象成数组。今天往前一步:当你手里不是一只股票,而是一篮子十只、几百只股票时,它们天然就是一张二维表,也就是「矩阵」。处理矩阵有一套专门的工具,叫线性代数。听到「线性代数」四个字先别怕,我保证今天不推一个公式让你死记,全部用大白话加生活类比讲清楚。这一节做五件事 — 第一,讲矩阵乘法这个符号 @,它能把「一篮子股票按权重组合起来」这件事用一行代码搞定;第二,讲怎么用 linalg 解方程,顺手算出一个「波动最小的组合」;第三,讲协方差矩阵,它描述一篮子股票是「一起涨一起跌」还是「各走各的」;第四,讲特征分解,把这一团乱麻拆成几个干净的「涨跌方向」;第五,也是最神奇的,讲 PCA 主成分分析 — 它能从十只看似不同的股票里,挖出一个看不见的「市场大盘因子」。学完你会明白,为什么散户买十只票自以为分散了风险,其实可能都在同一条船上。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 搞懂矩阵乘法符号 @ — 它跟逐元素乘的星号完全是两回事,@ 是「一篮子股票按权重加权求和」,组合收益一行代码算完
- 02 学会用 `linalg.solve` 解线性方程组 — 求「波动最小的组合权重」本质就是解一道方程,这是量化里最常见的优化雏形
- 03 理解协方差矩阵和相关矩阵 — 一张表看清一篮子股票是抱团齐涨跌还是各走各路,对角线是各自波动、非对角是两两联动
- 04 掌握特征分解 — 把一团互相纠缠的股票涨跌,拆成几个互不打架的「独立方向」,最大那个方向就是最强的共同走势
- 05 看懂 PCA 主成分分析 — 从十只股票里挖出看不见的「市场因子」,理解为什么 A 股个股七成以上跟着大盘走,所谓分散持仓常常是假分散

02 · CONTEXT · 历史与背景

# 买了十只票还是一起亏,老王的「假分散」

// 不读历史的策略走不远

讲一个散户老王的真实困惑。老王炒股5年,听人说「不要把鸡蛋放在一个篮子里」,于是他很认真地分散 — 买了茅台、招行、比亚迪、万科、海康威视等等十只股票,横跨白酒、银行、新能源、地产、科技五个完全不同的行业。他心想:这下稳了,一个行业崩了还有别的顶着。

结果 2022 年大盘下跌,老王打开账户一看 — 十只票齐刷刷全是绿的,没有一只逆势上涨。他百思不得其解:明明买的是完全不同的行业,怎么会一起跌?这不科学啊。

问题出在哪?老王犯了一个几乎所有散户都会犯的错 — 他以为「行业不同」就等于「风险分散」,但他忽略了一件事:这十只票背后,都被同一个看不见的力量牵着走,那就是「市场大盘」。当大盘情绪好,资金涌入,十只票一起涨;当大盘恐慌,资金撤离,十只票一起跌。行业的差别在这种系统性的大涨大跌面前,几乎可以忽略。

这个「看不见的力量」,在数学上有个精确的描述方法,就是今天要讲的 PCA 主成分分析。如果老王早点学会 PCA,他会把这十只股票的历史涨跌数据丢进去,机器会告诉他:这十只票 60% 到 70% 的涨跌,都由同一个「第一主成分」决定,而这个第一主成分,几乎就是沪深 300 大盘指数本身。换句话说,他买十只票,本质上跟买一只沪深 300 指数基金差不多,所谓的分散是假分散。

那真正的分散该怎么做?这就要用到协方差矩阵和最小方差组合 — 找出那些彼此联动最弱、甚至反向走的资产搭配在一起,比如长江电力这种防御股跟比亚迪这种进攻股的组合,才是真正降低波动的分散。

今天我们就用真实的十只 A 股5年数据,亲手算一遍:相关矩阵看谁跟谁抱团、特征分解找出市场因子、PCA 揭穿假分散。学完这一节,你看一篮子股票的眼光,会从「散户级」直接升到「量化研究员级」。

03 · CORE CONCEPTS · 核心概念详解

## 把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

### CONCEPT\_01

#### 矩阵乘法符号 @ — 一篮子股票按权重组合,一行搞定

先说一个最容易踩的坑:在 NumPy 里,星号 `*` 和 `@` 是两个完全不同的操作。星号是「逐元素相乘」,两个同样形状的数组对应位置各乘各的。而 `@` 是「矩阵乘法」,做的是「对应相乘再求和」。新手最常见的 bug 就是该用 `@` 的地方写了星号,结果数字全错还找不到原因。

那矩阵乘法到底在算什么?用组合收益来理解最直观。假设你有十只股票,今天每只涨跌幅是一个数,排成一行就是一个长度 10 的向量。你又给每只股票分配了一个仓位权重,比如各放一成,也是一个长度 10 的向量。你想知道「整个组合今天涨跌多少」,做法是:每只股票的涨跌幅乘以它的权重,再把十个乘积加起来。这个「对应相乘再求和」的操作,正是矩阵乘法 `@` 干的事。一行代码 `组合收益 等于 收益 @ 权重` 就算完了。

再升一级。如果你不止今天一天,而是过去 1200 个交易日,每天十只股票的涨跌排成一张 1200 行、10 列的大表,这就是一个矩阵。这张收益矩阵 `@` 权重向量,一次就能算出 1200 天每天的组合收益,完全不用写循环。这就是为什么说「金融组合天然就是矩阵乘法」——把数据摆成矩阵,用 `@` 一次性算完,既快又不容易错。

一句话记住:逐个对应相乘用星号,加权求和、组合、投影这类「相乘再相加」的统统用 `@`。写之前先想清楚要哪一个。

### CONCEPT\_02

#### 解线性方程组 `linalg.solve` — 求最优组合就是解一道方程

线性代数的另一大用处是解方程组。中学时我们解过 2 元一次方程组,两个未知数两个等式。当未知数变成十个、上百个,手算就不可能了,这时就用 NumPy 的 `linalg.solve`。你给它一个系数矩阵 `A` 和一个结果向量 `b`,它一行就把满足 `A 乘 x 等于 b` 的那组未知数 `x` 解出来。

这在量化里太有用了,因为很多「找最优」的问题,本质都是在解方程。今天我们用它干一件具体的事:找「波动最小的组合」。给定一篮子股票,怎么分配权重,能让整个组合的涨跌最平稳、波动最小?这个问题有个经典的数学答案,需要用到协方差矩阵的逆,而求逆和解方程是一回事。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT\_03

## 协方差矩阵与相关矩阵 — 一张表看清谁跟谁抱团

现在引入今天的核心数据结构:协方差矩阵。它是一张方方正正的表,行和列都是你的那一篮子股票。十只股票,就是一张十行十列的表。这张表回答一个问题:任意两只股票,是倾向于一起涨一起跌,还是各走各的?

表的对角线上,是每只股票「自己跟自己」的协方差,也就是它自身波动的大小 — 波动越大的股票,这个数越大。表的非对角线上,是「两只不同股票之间」的协方差 — 数值为正,说明这俩倾向于同涨同跌;为负,说明一个涨另一个常常跌,是天然的对冲搭档;接近零,说明两者基本没关系。

协方差有个小麻烦:它的数值大小受股票本身波动幅度影响,不同股票之间不好直接比。所以我们常把它「标准化」成相关系数,做出相关矩阵。相关系数永远在负一到正一之间:正一是完全同步,负一是完全相反,零是毫无关系。这样十只票两两之间的关系,一张相关矩阵热图就一目了然 — 红成一片说明大家高度抱团,花花绿绿说明各有各的脾气。

对散户的实战意义太大了:你以为买了不同行业就分散了,把它们的相关矩阵一画,如果发现两两相关系数普遍在 0.5 以上,那就是抱团,根本没分散。真正的分散,要去找那些相关系数低、甚至为负的搭配。

CONCEPT\_04

## 特征分解 — 把一团乱麻的涨跌拆成几个干净方向

协方差矩阵虽然信息全,但十只股票两两纠缠,看起来像一团乱麻。特征分解就是把这团乱麻「梳直」的工具。

打个比方。一群人在广场上走动,看起来杂乱无章。但如果你站高一点看,可能发现其实大部分人都在朝同一个大方向移动(比如散场往出口走),只有少数人在做别的小动作。特征分解干的就是这件事:它把矩阵里所有的波动,分解成几个互相垂直、互不打架的「方向」,每个方向叫一个「特征向量」,每个方向上波动的强弱叫「特征值」。特征值越大,说明那个方向上的共同波动越强。

对一篮子股票的协方差矩阵做特征分解,最大特征值对应的那个方向,往往就是「所有股票一起涨一起跌」的方向 — 这正是大盘整体的涨跌。第二大的方向可能是「成长股涨、价值股跌」之类的风格切换。后面那些特征值很小的方向,就是个股自己的小脾气。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT\_05

## PCA 主成分分析 — 挖出看不见的「市场因子」,揭穿假分散

把前面几块拼起来,就是今天的高潮:PCA,主成分分析。它是量化里使用频率最高的「降维」工具之一,听起来高深,其实就是「特征分解的实战应用」。

PCA 干的事一句话说清:从一大堆互相关联的数据里,自动找出几个最能概括全局的「主方向」,叫主成分。对一篮子股票来说,做法是:先把每只股票的收益标准化(消除波动量级差异,这一步很关键,忘了做的话第一主成分会被波动最大的那只股票带偏),再算相关矩阵,再做特征分解,排在第一的那个方向就是「第一主成分」。

神奇的地方来了:对 A 股一篮子股票做 PCA,第一主成分通常能解释 30%~40% 甚至更多的总波动,而且每只股票在这个方向上的「载荷」(也就是暴露程度)几乎全是正的、数值相近。这说明什么?说明有一股共同的力量,同时拉动所有股票一起动 — 这股力量就是「市场大盘因子」。我们可以把第一主成分还原成一条净值曲线,跟真实的沪深 300 一比,会发现它俩几乎重合,相关系数很高。这就用数据证明了:那个看不见的市场因子,真实存在,而且就是大盘。

这正是老王「假分散」的根源:他的十只票,大部分涨跌都被这同一个第一主成分支配,所以才会齐涨齐跌。PCA 的价值,就是把这种肉眼看不见的「共同命运」量化出来,让你心里有数 — 想真分散,就得找那些在第一主成分上载荷低、或者在别的主成分上唱反调的资产。

最后补一个冷知识:还有一个叫 SVD(奇异值分解)的工具,直接对收益数据做分解,能得到跟特征分解几乎一模一样的主成分,殊途同归。很多库内部用的是 SVD,因为它数值上更稳。你只要知道「PCA、特征分解、SVD 三者血脉相连」就够了,具体哪个更稳,交给库去操心。

04 · PYTHON LAB · 真实可跑代码

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分 解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_033_linalg_pca.py - NumPy 进阶:矩阵运算 + 协方差特征分解 + PCA
# 用真实A股10只不同行业蓝筹5年日线,演示五件事:
# ① @ 矩阵乘法算组合收益 ② linalg.solve 解最小方差组合权重
# ③ 协方差/相关矩阵 ④ 特征分解 ⑤ PCA 找出看不见的「市场因子」
# 数据:baostock(免费、稳定、国内零翻墙;指数不复权,个股前复权)
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

np.random.seed(42)
START = '2020-01-01'
END = '2024-12-31'

# ===== 0. 数据拉取(baostock)=====
print('==== 0. 数据拉取(baostock)====')
lg = bs.login()
if lg.error_code != '0':
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
print(f'baostock 登录:{lg.error_msg}')

def bs_get(code, name, adjust='2'):
    """拉日线收盘价 close。adjust 3=不复权 / 2=前复权 / 1=后复权"""
```



04 · PYTHON LAB · 真实可跑代码 · 续 (2/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分 解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
rs = bs.query_history_k_data_plus(
code, 'date,close', start_date=START, end_date=END,
frequency='d', adjustflag=adjust)
rows = []
while (rs.error_code == '0') and rs.next():
rows.append(rs.get_row_data())
df = pd.DataFrame(rows, columns=['date', 'close'])
df['date'] = pd.to_datetime(df['date'])
df['close'] = df['close'].astype(float)
df = df.set_index('date')
print(f' {name:<6} {code} {len(df)} 条 {df.index[0].date()} →
{df.index[-1].date()}')
return df['close']

# 沪深 300 指数做基准(指数无复权概念,用不复权 adjust=3)
hs300 = bs.get('sh.000300', '沪深300', adjust='3')

# 10 只不同行业蓝筹(前复权 adjust=2,分红送股已还原)
tickers = {
'茅台': 'sh.600519', # 白酒
'招行': 'sh.600036', # 银行
'中国平安': 'sh.601318', # 保险
'长江电力': 'sh.600900', # 公用事业(防御股)
'比亚迪': 'sz.002594', # 汽车
'宁德时代': 'sz.300750', # 电池新能源
'万科A': 'sz.000002', # 房地产
'恒瑞医药': 'sh.600276', # 医药
'中国石化': 'sh.600028', # 能源
'海康威视': 'sz.002415', # 科技
}

# 铁律 39:多 ticker 必须用 dict 显式映射,绝不能 df.columns = list(...) 赋值
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
prices = {name: bs_get(code, name, adjust='2') for name, code in tickers.items()}
price_df = pd.DataFrame(prices)
bs.logout()

# 个股跟指数按日期内连接对齐, 去掉任何含 NaN 的行
all_df = price_df.join(hs300.rename('沪深300'), how='inner').dropna()
stock_cols = list(tickers.keys())
print(f'对齐后 {all_df.index[0].date()} → {all_df.index[-1].date()}, {len(all_df)}
个交易日, {len(stock_cols)} 只股票')

# 日收益率矩阵 R: 每行一天, 每列一只股票 (去掉首行 NaN)
rets = all_df[stock_cols].pct_change().dropna()
mkt_ret = all_df['沪深300'].pct_change().reindex(rets.index)
R = rets.values # ndarray 形状 (T, N)
T, N = R.shape
print(f'收益率矩阵 R 形状 = {R.shape} (T={T} 天, N={N} 只)')

# ===== 1. @ 矩阵乘法: 一行算出组合收益 =====
print('\n=== 1. @ 矩阵乘法: 组合收益 ===')
w_eq = np.ones(N) / N # 等权重向量, 每只 1/10

# 矢量化: 收益矩阵 (T, N) @ 权重向量 (N, ) = 组合日收益 (T, )
t0 = time.perf_counter()
port_ret_matmul = R @ w_eq
t_matmul = time.perf_counter() - t0

# 对照组: 纯 Python 双重 for 循环算同样的东西
t0 = time.perf_counter()
port_ret_loop = np.empty(T)
for ti in range(T):
    s = 0.0
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分 解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
for ni in range(N):
    s += R[ti, ni] * w_eq[ni]
    port_ret_loop[ti] = s
    t_loop = time.perf_counter() - t0

max_diff = np.abs(port_ret_matmul - port_ret_loop).max()
print(f'@ 矩阵乘法用时 {t_matmul*1000:.3f} ms')
print(f'for 循环用时 {t_loop*1000:.3f} ms')
print(f'结果最大差异 = {max_diff:.2e} (几乎为 0, 两种算法等价)')
print(f'矩阵乘法加速 ≈ {t_loop/max(t_matmul, 1e-9):.0f} 倍')

# ===== 2. linalg.solve: 解出最小方差组合权重 =====
print('\n==== 2. linalg: 最小方差组合 ====')
# 年化协方差矩阵 Σ (日度协方差 × 252)
cov = np.cov(R, rowvar=False) * 252
ones = np.ones(N)
# 最小方差权重公式  $w = \Sigma^{-1} \cdot 1 / (1^T \cdot \Sigma^{-1} \cdot 1)$ 
# 用 solve 解线性方程  $\Sigma \cdot x = 1$ , 再归一化, 比 inv 更快更数值稳定
x = np.linalg.solve(cov, ones)
w_mv = x / x.sum()

def port_vol(w, cov):
    """组合年化波动率 = sqrt( $w^T \Sigma w$ )"""
    return float(np.sqrt(w @ cov @ w))

vol_eq = port_vol(w_eq, cov)
vol_mv = port_vol(w_mv, cov)
print('最小方差权重:')
for name, wi in sorted(zip(stock_cols, w_mv), key=lambda kv: -kv[1]):
    print(f' {name:<6} {wi*100:6.1f}%')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'等权组合年化波动 = {vol_eq*100:.1f}%')
print(f'最小方差组合年化波动 = {vol_mv*100:.1f}% (降低 {(1-vol_mv/vol_eq)*100:.0f}%)')

# ===== 3. 相关系数矩阵 (热图) =====
print('\n==== 3. 相关矩阵 ====')
corr = np.corrcoef(R, rowvar=False)
off = corr[np.triu_indices(N, k=1)]
print(f'非对角相关系数 平均 {off.mean():.2f} 最高 {off.max():.2f} 最低 {off.min():.2f}')

fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(corr, cmap='RdYlGn_r', vmin=-1, vmax=1)
ax.set_xticks(range(N)); ax.set_xticklabels(stock_cols, rotation=45, ha='right')
ax.set_yticks(range(N)); ax.set_yticklabels(stock_cols)
for i in range(N):
    for j in range(N):
        ax.text(j, i, f'{corr[i,j]:.2f}', ha='center', va='center', fontsize=7)
ax.set_title('10 只 A 股日收益相关系数矩阵(2020-2024)')
fig.colorbar(im, ax=ax, fraction=0.046)
fig.tight_layout(); fig.savefig('chart_corr.png', dpi=130); plt.close(fig)

# ===== 4. 特征分解: 把协方差拆成独立的涨跌方向 =====
print('\n==== 4. 特征分解 ====')
# 对相关矩阵做特征分解 (对称矩阵用 eigh, 保证实数 + 升序)
eigvals, eigvecs = np.linalg.eigh(corr)
order = np.argsort(eigvals)[::-1] # 反转成降序
eigvals = eigvals[order]
eigvecs = eigvecs[:, order]
explained = eigvals / eigvals.sum() # 每个主成分解释的方差比例
cum_explained = np.cumsum(explained)
print('各主成分解释方差比例:')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
for i in range(N):
    print(f' PC{i+1}: 特征值 {eigvals[i]:.2f} 解释 {explained[i]*100:5.1f}% 累计 {cum_explained[i]*100:5.1f}%')

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(range(1, N+1), explained*100, color='#1f77b4', label='单个主成分')
ax.plot(range(1, N+1), cum_explained*100, 'o-', color='#d62728', label='累计')
ax.set_xlabel('主成分序号'); ax.set_ylabel('解释方差比例 (%)')
ax.set_title('碎石图:前几个主成分就解释了大部分共同波动')
ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_scee.png', dpi=130); plt.close(fig)

# ===== 5. PCA:第一主成分就是看不见的「市场因子」 =====
print('\n==== 5. PCA 市场因子 ====')
pc1_load = eigvecs[:, 0].copy() # 第一主成分载荷(每只股票的暴露)
if pc1_load.mean() < 0: # 统一符号,让载荷整体为正便于解读
    pc1_load = -pc1_load

# PC1 时间序列 = 标准化收益 @ 第一特征向量
Rz = (R - R.mean(axis=0)) / R.std(axis=0)
pc1_ts = Rz @ pc1_load
corr_pc1_mkt = float(np.corrcoef(pc1_ts, mkt_ret.values)[0, 1])
print(f'第一主成分解释方差 {explained[0]*100:.1f}%')
print(f'PC1 与沪深 300 日收益相关系数 = {corr_pc1_mkt:.2f}(越接近 1 越说明 PC1 就是大盘因子)')
print('第一主成分载荷(每只股票在「市场因子」上的暴露):')
for name, ld in sorted(zip(stock_cols, pc1_load), key=lambda kv: -kv[1]):
    print(f' {name:<6} {ld:.2f}')

fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#2ca02c' if v >= 0 else '#d62728' for v in pc1_load]
ax.bar(stock_cols, pc1_load, color=colors)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/7)

# NumPy 矩阵运算五件套 — @ 组合收益 / linalg 最小方差 / 协方差相关矩阵 / 特征分解 / PCA 市场因子

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

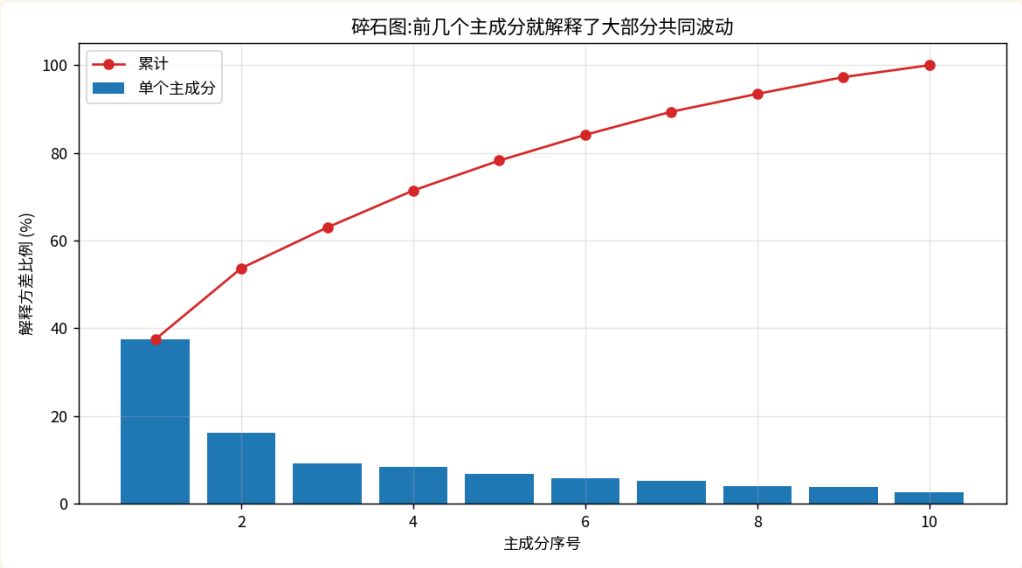
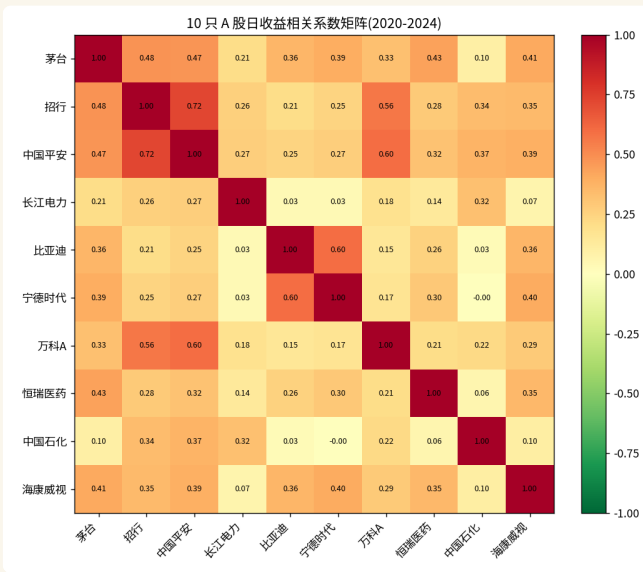
```
ax.set_ylabel('PC1 载荷'); ax.set_title('第一主成分载荷:谁更随大盘起舞')
ax.set_xticklabels(stock_cols, rotation=45, ha='right'); ax.grid(alpha=0.3,
axis='y')
fig.tight_layout(); fig.savefig('chart_pc1_load.png', dpi=130); plt.close(fig)

# PC1 重构的「市场因子」累计净值 vs 真实沪深 300 净值
scale = mkt_ret.std() / pc1_ts.std() # 把 PC1 缩放到跟指数同量级
pc1_ret = pc1_ts * scale
nav_pc1 = (1 + pd.Series(pc1_ret, index=rets.index)).cumprod()
nav_mkt = (1 + mkt_ret).cumprod()
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(nav_mkt.index, nav_mkt.values, label='真实沪深 300', color='#1f77b4',
lw=2)
ax.plot(nav_pc1.index, nav_pc1.values, label='PCA 第一主成分(市场因子)',
color='#ff7f0e', lw=1.5, ls='--')
ax.set_title(f'看不见的市场因子 vs 真实大盘(相关系数 {corr_pc1_mkt:.2f})')
ax.set_ylabel('累计净值'); ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_pc1_vs_mkt.png', dpi=130); plt.close(fig)

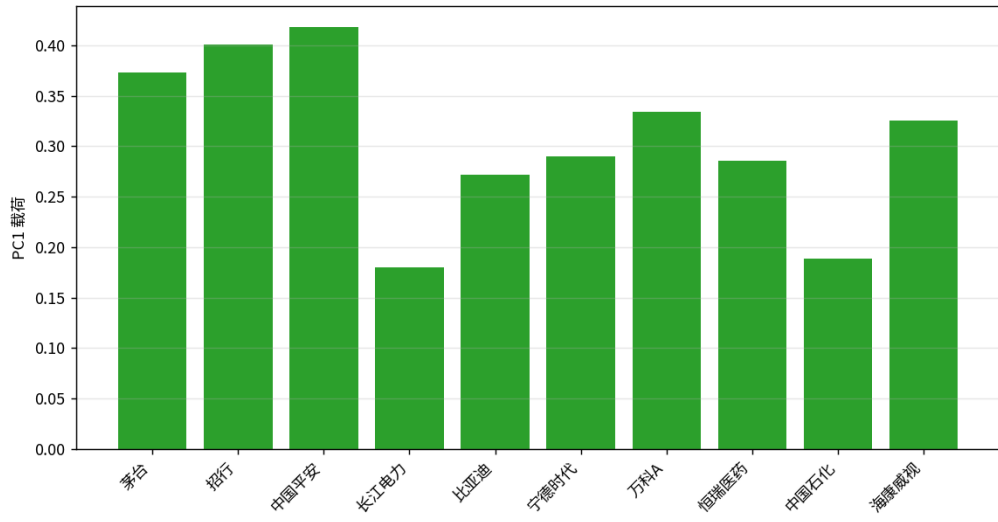
# ===== 6. SVD 殊途同归(验证) =====
print('\n==== 6. SVD 与特征分解殊途同归 ====')
# 对标准化收益直接做 SVD, 第一右奇异向量应跟第一特征向量方向一致
U, S, Vt = np.linalg.svd(Rz, full_matrices=False)
svd_pc1 = Vt[0].copy()
if svd_pc1.mean() < 0:
    svd_pc1 = -svd_pc1
align = float(np.abs(np.corrcoef(svd_pc1, pc1_load)[0, 1]))
print(f'SVD 第一主方向与特征分解第一主方向的一致度 = {align:.3f}(接近 1 = 两条路同一个答案)')
print('\n[done] 5 张图已保存,output.txt 见上方全部打印')
```

# 回测结果

// · matplotlib 真实输出



第一主成分载荷:谁更随大盘起舞



看不见的市场因子 vs 真实大盘(相关系数 0.92)





矩阵乘法 @ vs for 循环

@ 用 0.015 ms, 纯 Python 双重 for 用 1.390 ms, 加速约 94 倍, 结果最大差异  $1.4e-17$  (完全等价)

最小方差组合  
(linalg.solve)

重仓防御股: 长江电力 45.2% + 中国石化 28.0%; 年化波动从等权的 20.1% 降到 14.9%, 降低 26%

相关系数矩阵

10 只两两相关平均 0.29; 最高招商银行-中国平安 0.72; 比亚迪-宁德时代 0.60; 长江电力/中国石化跟谁都低 (0.0~0.3)

特征分解 / 碎石图

第一主成分特征值 3.75 解释 37.5%, 第二主成分 16.2%, 前两个累计 53.7%

PCA 第一主成分 = 市场因子

PC1 解释 37.5% 总波动, 与真实沪深 300 日收益相关系数 0.92; 载荷全为正, 中国平安 0.42 最高、长江电力 0.18 最低

SVD 与特征分解一致性

对标准化收益做 SVD, 第一主方向与特征分解第一主方向一致度 1.000 (殊途同归)

ANALYSIS · 这张图告诉我们什么

► 01 市场因子主导大方向, 但只解释约四成

第一主成分只解释 37.5% 却与沪深 300 相关高达 0.92 — 说明 A 股存在一个强势的共同「市场因子」主导大方向, 但个股仍保留约六成自己的脾气, 这正是「分散有用但有限」的数据证据。

► 02 防御股才是真分散器: PCA 与最小方差殊途同归

最反直觉的交叉验证: 在第一主成分上载荷最低的两只防御股 (长江电力 0.18、中国石化 0.19), 恰恰是最小方差组合给出最大权重的两只 (45.2%、28.0%)。PCA 和组合优化从两个完全不同的角度, 共同指认「跟大盘联动最弱的资产才是真正的分散器」。

► 03 同板块抱团: 招商银行-平安 0.72、比亚迪-宁德 0.60

同板块抱团一目了然: 招商银行-中国平安 0.72、比亚迪-宁德时代 0.60。散户买十只不同行业票自以为分散, 但金融板块内部、新能源板块内部高度同涨同跌, 真实分散度远低于股票数量给人的错觉。

► 04 无约束最小方差会做空高相关资产 (需加非负约束)

最小方差组合出现负权重 (招商银行 -1.3%、中国平安 -2.6%) — 无约束优化会做空高相关资产来对冲, 实盘必须加非负约束, 这是教学演示与可落地策略的关键差别。

► 05

## 矩阵乘法比双重 for 快约 94 倍且结果完全等价

矩阵乘法 @ 比手写双重 for 循环快约 94 倍且结果完全一致(差异仅  $1.4\text{e-}17$  的浮点误差)— 再次印证矢量化思维:能用矩阵运算就别写循环。

05 · REAL MARKETS · 真实市场案例

# A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

## A 股散户

茅台 / 招行 / 比亚迪 / 万科 / 海康威视 等十只跨行业蓝筹

最典型的散户场景:买十只不同行业的票自以为分散了风险。把这十只票的历史日收益做 PCA,第一主成分往往解释 40% 上下的总波动,且每只票载荷都为正 — 证明它们大部分时候是齐涨齐跌的「假分散」。真正想降低波动,得看相关矩阵去找低相关甚至负相关的搭配,而不是简单堆数量。这一节的代码就是把这个分析完整跑一遍。

## 公募 / 银行理财

「稳健型」「低波动」组合

支付宝、银行 App 里那些标着「稳健」「低波动」的组合产品,背后大量用到协方差矩阵和最小方差优化。基金经理把候选资产的协方差矩阵算出来,用类似今天 `linalg.solve` 的方法,解出一组让整体波动最小的权重。普通人看到的是「稳健」两个字,背后其实就是今天这套线性代数。

## 量化私募

全市场几千只股票 × 上百个因子

量化私募研究全市场几千只股票时,会有成百上千个因子(估值、成长、动量等),彼此高度相关、信息冗余。研究员用 PCA 把上百个因子压缩成几十个互不重叠的「主因子」,既去掉噪音又大幅提速。PCA 降维是因子工程的标准操作,今天学的就是它的最小可运行版本。

## 指数编制

沪深 300 / 中证 500

宽基指数本质上就是市场「第一主成分」的一种近似实现 — 按市值加权把一篮子大票打包,涨跌反映的正是那个市场大盘因子。所以我们才能用 PCA 的第一主成分去跟沪深 300 对比验证:两条曲线高度重合,说明指数确实抓住了市场最主要的共同波动。

## 券商风控

自营盘 / 两融组合压力测试

券商风控系统每天对持仓组合做特征分解,找出「组合最大的风险暴露方向」(最大特征值对应的方向)。一旦发现组合在某个方向上过度集中,就会预警。压力测试里常问的「如果市场系统性下跌 10%,我会亏多少」,算的就是组合在第一主成分方向上的暴露。今天的特征分解,正是这套风控的数学地基。

## 这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

### △ 01

#### 星号 和 @ 搞混 — 数字全错还找不到原因

这是矩阵运算第一大坑。星号是逐元素相乘,@ 是矩阵乘法,两者完全不同。该算加权组合却写了星号,得到的是一张二维表而不是一个组合收益,数字看着像那么回事但全错。**正确做法:**凡是「相乘再求和」「加权」「投影」「组合」,一律用 @;只有「两个同形状数组对应位置各乘各的」才用星号。写之前先在脑子里过一遍要的是哪种。

### △ 02

#### 矩阵形状不匹配 — 一报错就懵

矩阵乘法对形状有严格要求:前一个矩阵的列数,必须等于后一个的行数。形状对不上直接报错。新手看到 shapes not aligned 之类的报错往往一脸懵。**正确做法:**做矩阵运算前,先把两个数组的 shape 都 print 出来看一眼,确认中间那两个维度对得上。八成的矩阵报错,看一眼形状就解决了。这是铁律级的好习惯。

### △ 03

#### 用 inv 求逆而不用 solve — 数值爆炸

数学公式里写着矩阵求逆,很多人就照着写 inv。但当一篮子股票里有几只走势特别像时,协方差矩阵接近「奇异」,直接求逆会算出一堆离谱的大数,权重变得荒唐(比如某只票权重高达 +800%、另一只 -700%)。**正确做法:**能用 np.linalg.solve 解方程的,绝不用 inv 再做乘法。solve 更快、数值更稳,是工程上的标准选择。

### △ 04

#### 特征值顺序拿反 — 取了最弱方向当最强

np.linalg.eigh 返回的特征值是从小到大升序排列的。如果你直接取第一个,拿到的是最小特征值、最弱的方向,跟你想要的「最强共同方向」正好相反。**正确做法:**eigh 拿到结果后,用排序反转一下,把最大特征值排到最前面,再取第一主成分。一定要核对一下第一主成分的特征值是不是最大的那个。

### △ 05

#### PCA 前忘了标准化 — 第一主成分被高波动股绑架

不同股票波动量级差很多(小盘股可能是大盘股的好几倍)。如果直接对原始收益做 PCA,第一主成分会被那只波动最大的股票主导,反映的是「那只票的脾气」而不是「市场共同走势」,结论全错。**正确做法:**PCA 前先把每只股票的收益做标准化(减均值除以标准差,也就是 z-score),让大家站在同一起跑线,再做特征分解,这样第一主成分才真正代表共同因子。

# NumPy 矩阵运算与 PCA 实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 逐元素相乘用星号,加权求和、组合、投影用 @ — 写之前先想清楚要哪一个,这是第一大坑
- 02 做矩阵乘法前先 print 两个数组的 shape — 确认中间维度对得上,八成报错一眼解决
- 03 解方程、求最优权重用 `np.linalg.solve(A, b)`,绝不用 `inv(A) @ b` — 更快更数值稳定
- 04 日度协方差要年化记得乘 252 — 算年化波动、年化风险前先确认时间尺度统一
- 05 对称矩阵(协方差/相关矩阵)特征分解用 `eigh` 不用 `eig` — 更快且保证实数特征值
- 06 `eigh` 返回升序,要主成分必须先反转成降序 — 核对第一主成分的特征值是最大的那个
- 07 PCA 前一定先把每列标准化(z-score)— 否则第一主成分会被波动最大的股票绑架

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

## 07 · RECAP · 一页课件总结

# 把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 矩阵乘法 @ 跟逐元素的星号完全是两回事 — @ 做「相乘再求和」, 组合收益 等于 收益矩阵 @ 权重向量, 一行算完不用循环
- 02 解线性方程组用 linalg.solve — 求最小方差组合权重本质就是解方程, 能用 solve 别用 inv(更快更稳)
- 03 协方差矩阵/相关矩阵 — 对角线是各自波动, 非对角是两两联动; 相关系数在负一到正一之间, 一张热图看清谁跟谁抱团
- 04 特征分解把纠缠的涨跌拆成几个独立方向 — 最大特征值方向就是最强共同走势; 对称矩阵用 eigh, 记得反转成降序
- 05 PCA 主成分分析 — 标准化后做特征分解, 第一主成分通常解释 30%~40% 总波动, 就是看不见的「市场大盘因子」
- 06 散户的「假分散」 — 买十只不同行业票若第一主成分占比高、载荷全为正, 本质是齐涨齐跌, 跟买一只指数差不多
- 07 PCA、特征分解、SVD 血脉相连 — 三者殊途同归, 库内部常用数值更稳的 SVD, 你理解到这层就够

## 08 · SELF-CHECK · 自测题

## 答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

**Q01** NumPy 里星号 `*` 和 `@` 有什么区别?算「一篮子股票按权重组合的收益」该用哪个?为什么?(提示:星号逐元素、`@` 相乘再求和;组合收益是加权求和用 `@`)

**Q02** 为什么求最小方差组合权重时,工程上推荐用 `np.linalg.solve` 而不是先 `inv` 再相乘?(提示:更快;协方差接近奇异时 `inv` 数值爆炸,`solve` 更稳)

**Q03** 协方差矩阵的对角线和非对角线分别代表什么?相关系数为负说明两只股票什么关系?(提示:对角是各自波动、非对角是两两联动;为负是反向、天然对冲)

**Q04** 对协方差矩阵做特征分解,最大特征值对应的方向通常代表什么?为什么协方差矩阵要用 `eigh` 而不是 `eig`?(提示:市场共同涨跌方向;协方差对称,`eigh` 更快且保证实数)

**Q05** 做 PCA 之前为什么必须先标准化每只股票的收益?如果不做会怎样?(提示:不同股票波动量级不同;不标准化第一主成分会被波动最大的那只股票主导,得出错误结论)

## 09 · NEXT STEP · 下一步

# 继续往前走

// 看完这节,下面是延伸方向 + 明天预告

## 推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Gilbert Strang 《Introduction to Linear Algebra》(2016 第 5 版,Wellesley-Cambridge)– 线性代数最经典的入门教材,MIT 公开课同款,矩阵乘法/解方程/特征分解讲得最透,配套视频免费
- ▶ Wes McKinney 《Python for Data Analysis》(2022 第 3 版,O'Reilly)– 附录 A 专讲 NumPy 高级数组操作与 linalg 模块,Pandas 作者亲笔,工程实战角度
- ▶ Jolliffe & Cadima 《Principal Component Analysis: a review and recent developments》(2016,Phil. Trans. R. Soc. A)– PCA 综述权威文献,从原理到应用一篇讲清,数学背景不深也能读
- ▶ Connor & Korajczyk 《Risk and Return in an Equilibrium APT》(1988,Journal of Financial Economics)– 把 PCA 用到资产定价、提取统计因子的开山论文,理解「市场因子」从何而来
- ▶ Python 工具栈 – NumPy 的 linalg 子模块(solve/eig/eigh/svd)、scipy.linalg(更全的分解)、scipy.sparse(稀疏矩阵省内存)、scikit-learn 的 PCA(一行 fit\_transform 的工业级封装),这四个是矩阵运算与降维的进阶生态

## NEXT LECTURE · 下一节预告

### DAY 034 Pandas Series/DataFrame

第 34 课进入 Pandas 的世界 — Series 和 DataFrame。今天我们一直在跟「纯数字矩阵」打交道,但真实的金融数据还带着日期、股票名这些标签。Pandas 的 DataFrame 就是一张「带行列标签的智能表格」,能自动按日期对齐、按名字取列,是量化日常用得最多的工具。明天我们亲手造一张上百只股票的行情表,讲透索引、自动对齐、loc 跟 iloc 的区别,从「裸数组」升级到「带标签的专业数据表」。